

# Chapter 4 Linked Lists

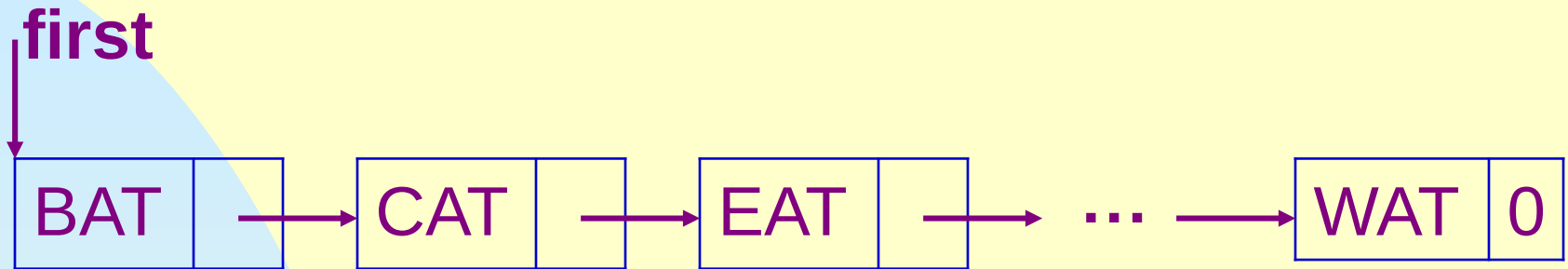
---

## 4.1 Singly linked lists and chains.

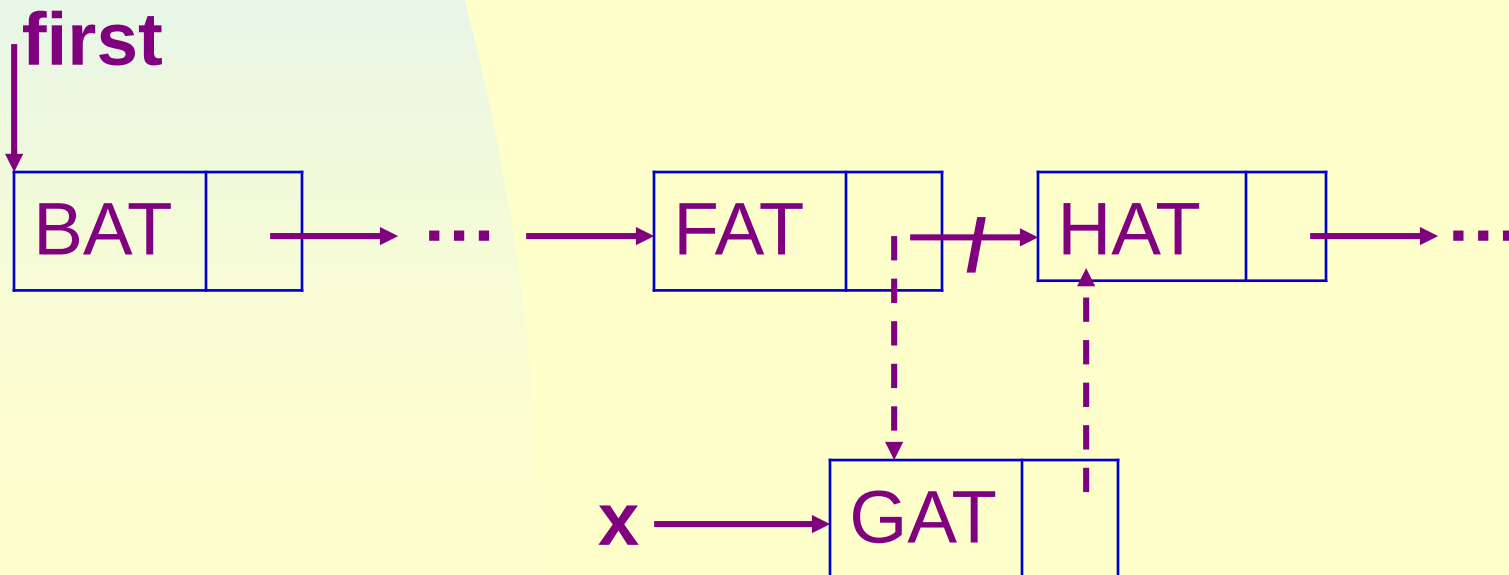
- The representation of **simple data structure (table, stack, queue)** using an **array** and a **sequential mapping** has the **properties**:
  - ✓ **Successive nodes** of the data object are stored at **Successively**.
  - ✓ This makes it easy to access an arbitrary node in  **$O(1)$** .

- **Disadvantage** of sequential mapping:
  - ✓ For ordered lists, **insertion and deletion** of arbitrary elements is **expensive**.
  - ✓ **For example**: Insert “GAT” into or delete “LAT” from the sequential list: (BAT, CAT, EAT, **FAT, HAT**, JAT, **LAT**, MAT, OAT, PAT, RAT, SAT, TAT, VAT, WAT)---need **data movement**.
- **Improvement---linked representation**:
  - ✓ Items of a list may be **placed anywhere** in the memory.
  - ✓ Each item is associated with a **point (link)** to the next item.

- ✓ The above **list** expressed in the **linked representation** is as follows:



- ✓ In linked list, **insertion (deletion)** of arbitrary elements does not need data movement:



- In general, a **linked list** is comprised of **nodes**; each node has zero or more data fields and **one or more link or pointer fields**.
- The linked structure of the figures in the above slide are called **singly linked lists** or **chains** in which **each node has exactly one pointer field**.
  - ✓ a **link** is used to go from one element to the next
  - ✓ list elements are stored in an **arbitrary order** in memory

## 4.2 Representing Chains in C++

### 4.2.1 Defining a **node** in C++

```
class ChainNode {  
private:  
    int data;  
    ChainNode *link;  
};
```

## 4.2 Representing Chains in C++

### 4.2.2 Designing a Chain class in C++

#### ■ Attempt 1 (without the definition of class Chain):

- ✓ **ChainNode \*f;**
- ✓ The **data members of the node** pointed to by **f** may be referenced in the following way: **f→data;**
- ✓ It will cause a **compile-time error** because a **private data member cannot be accessed** from outside of the class.

#### ■ Attempt 2:

- ✓ Making the **data members of ChianNode public** would do the trick, but that **violates data encapsulation**.
- ✓ So a solution might be to define **public member functions on ChainNode** to indirectly access the data members.

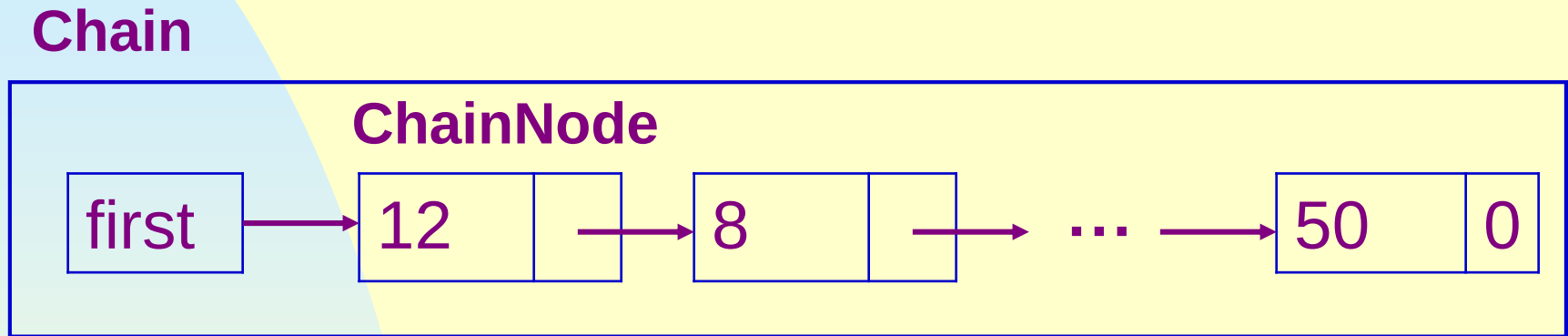
- ✓ **But this is not a good solution**, because this solution allows one to read and change these data members **from anywhere** in the program.
- ✓ The ideal solution would **only permit** those functions that perform **list manipulation operations (like inserting a node into or deleting a node from a chain)** access to the data members of ChainNode.

### ■ **Attempt 3:**

- ✓ Using **two classes: ChainNode and Chain**
  - The **Chain class** corresponds to the **entire chain**, which contains member functions that carry out list manipulation operation such as insert and delete.
  - The conceptual relationship between **ChainNode and Chain** are: an object of the class **Chain** consists of zero or more objects of the class **ChainNode**, or Chain **HAS-A** ChainNode

**Definition:** We say that a data object of **Type A** **HAS-A** data object of **Type B** if **A** conceptually **contains B** or **B is a part of A**.

**Chain HAS-A ChainNode.**



**Figure 4.7: Conceptual relationship** between Chain and ChainNode

HAS-A relationships between ADTs are usually expressed by making the contained class a number of the containing class. This is possible when the type A object contains **a fixed number** of type B objects.



- ✓ Figure 4.7 suggests that an object of class **Chain** **physically contains** many objects of class **ChainNode**; that is, **objects of ChainNode** are declared as **data members of Chain**.
- ✓ But the number of nodes in a chain is not a constant. So, it is **impossible** to know in advance the number of ChainNodes in Chain.
- ✓ For this reason, the **Chain class** is defined so that it **only contains the access pointer, first**, which points to **the first node** in the list.

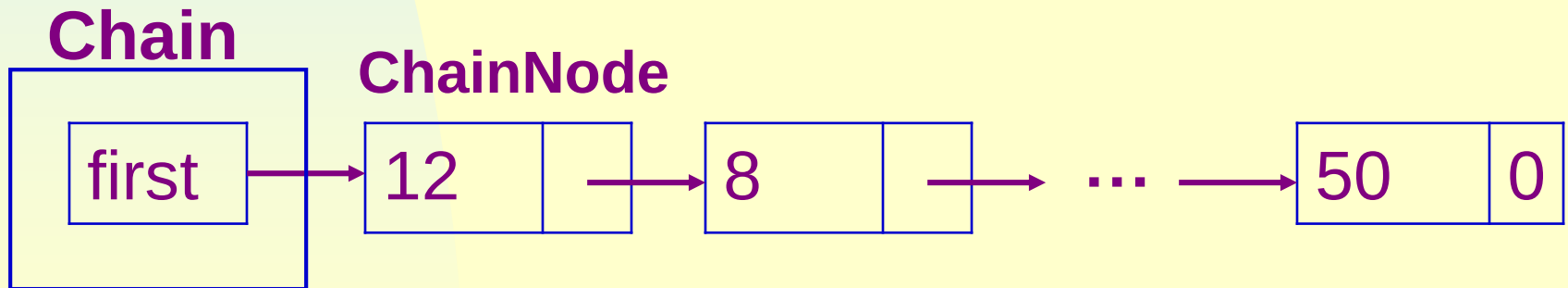


Figure 4.8: **Actual relationship** between Chain and ChainNode

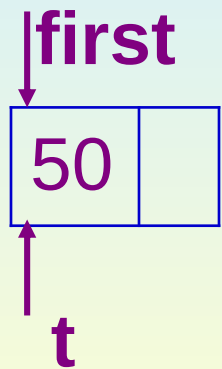
- **Question:** How to define the class Chain so that **only list manipulation operations** have access to the data members of the nodes ChainNode?
- ✓ **Approach 1:** by declaring **Chain to be a friend of ChainNode**. Only member functions of Chain and ChainNode can now access the private data members of ChainNode.
- ✓ **Approach 2:** using **nest classes**. Here, class ChainNode is defined inside the **private portion** of the definition of class Chain. So chainNode objects cannot be accessed outside class Chain. Notice that the data members of ChainNode are public. This ensures that they can be accessed by member functions of Chain.
- ✓ **Comparison:** Approach 1, the composite class approach **is preferred** over nest classed class approach, because a single node class **can be used by many different classes so long as each is a friend of the node class**.

```
class Chain; // forward declaration
class ChainNode {
friend class Chain;
    // to make functions of Chain be able to
    // access private data members of ChainNode
Public:
    ChainNode(int element = 0, ChainNode* next = 0)
        {data = element; link = next;}
private:
    int data;
    ChainNode *link;
};
class Chain {
public:
    // Chain manipulation operations
    ...
private:
    ChainNode *first;
};
```

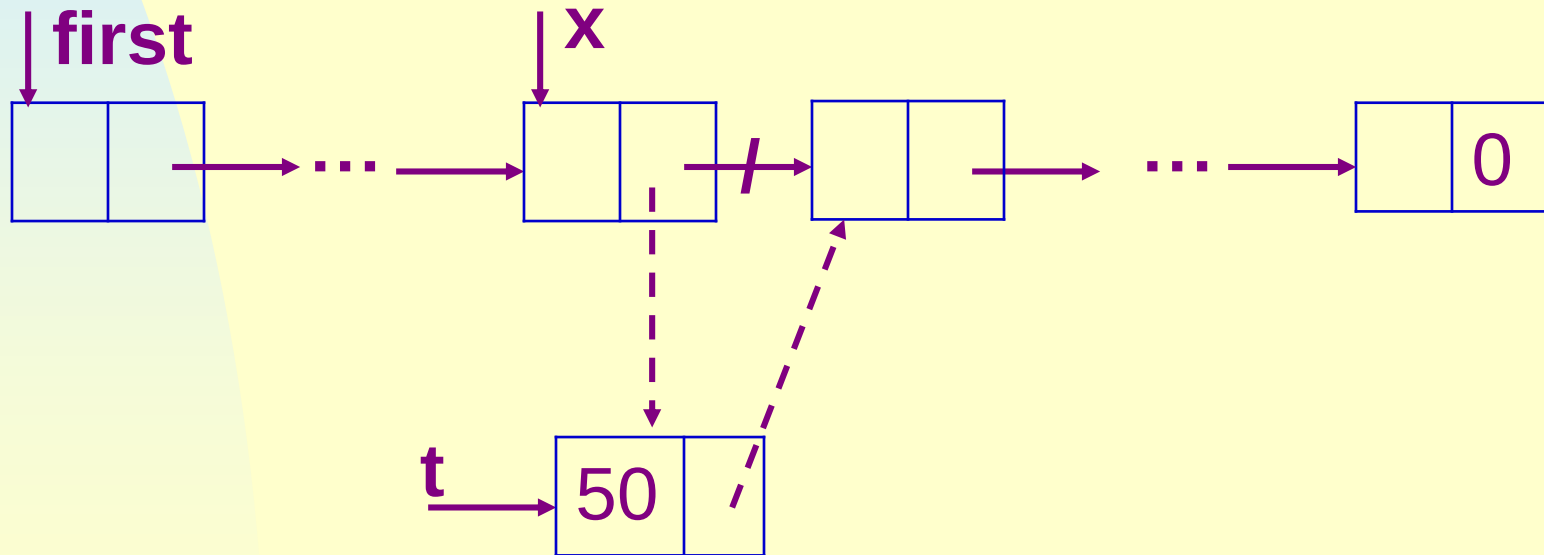
## 4.2.4 Chain Manipulation Operations

**Assume:** we use the **ChainNode** and **Chain** definitions in the above slide.

**Example 4.3:** **insert** a node with data field 50 **following** the node pointed by **x** except when the chain is empty.



(a)  $\text{first} == 0$

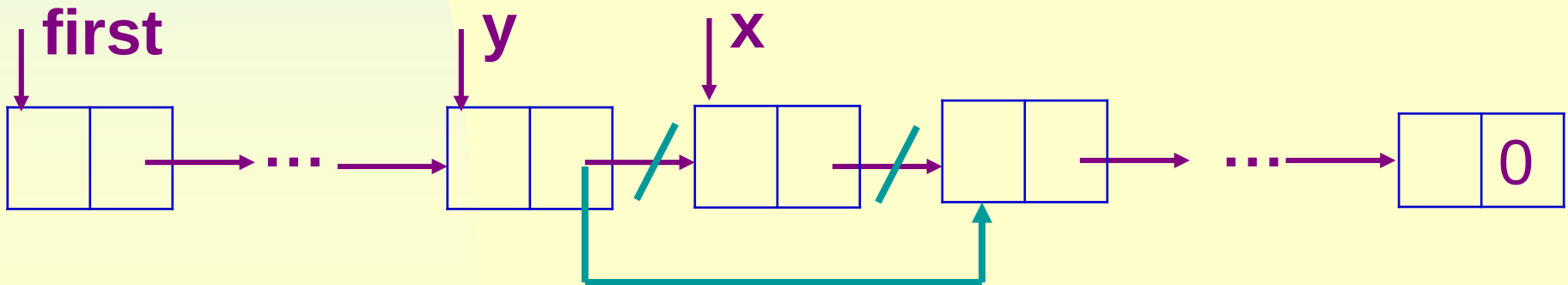


(b)  $\text{first} \neq 0$

```
void Chain::Insert50 (ChainNode *x)
{
    if (first)
        // insert after x
        x→link = new ChainNode(50, x→link);
    else
        // insert into empty chain
        first = new ChainNode(50);
}
```

**Example 4.4:** **deletes** node  $x$  from the chain. Let  $y$  point to the node (if any) that precedes  $x$ , and let  $y=0$  iff  $x==\text{first}$ .

```
void Chain::Delete (ChainNode * $x$ , ChainNode * $y$ )  
{  
    if ( $x==\text{first}$ )  $\text{first}=\text{first}->\text{link}$ ;  
    else  $y->\text{link}=x->\text{link}$ ;  
    delete  $x$ ;  
}
```



## 4.3 The Template Class Chain

We shall enhance the chain class of the previous section to make it more **reusable**.

### 4.3.1 Implementing Chains with Templates

```
template <class T> class Chain; // forward declaration
```

```
template <class T> class ChainNode {
```

```
friend class Chain<T>;
```

```
public:
```

```
    ChainNode(T element, ChainNode* next = 0)
```

```
    { data = element; link = next;}
```

```
private:
```

```
    T data;
```

```
    ChainNode *link;
```

```
};
```

```
template <class T>
```

```
class Chain {
```

```
public:
```

```
    Chain() { first=0;} // constructor initializing first to 0  
    // Chain manipulation operations
```

```
    ...
```

```
private:
```

```
    ChainNode<T> *first;
```

```
};
```

- Notice that **Chain<T>** has been made a **friend** of **ChainNode<T>**. This means that the members of **ChainNode<int>** can be accessed by members of **Chain<int>**, but not by members of **Chain<float>**.
- Similarly, notice that **first** is declared to be **a pointer to an object of type ChainNode<T>**. This ensures that an object of type **Chain<int>**, only consists of nodes of type **ChainNode<int>**.
  - ✓ A **empty chain of integers intchain** would be defined as:  
**Chain<int>** intchain;



## 4.3.2 Chain Iterators

- A **container class** is a class that represents a data structure that **contains or stores a number of data objects**. Objects can usually be added to or deleted from a container. (p.130)
- An **iterator** is an **object** that is used to access the elements of a **container class one by one**. (p.185)
- Iterators allow us to efficiently access and modify elements in a container without relying on the specific implementation details of the container.

## ● Why do we need an iterator?

Consider the following operations that one might perform on a container class  $C$ , all of whose elements are **integers**:

- (1) Output all integers in  $C$ .
- (2) Obtain the maximum, minimum, mean, median, or mode of all integers in  $C$ .
- (3) Obtain the sum, product, or sum of squares of all integers in  $C$ .
- (4) Obtain all integers in  $C$  that satisfy some property  $P$ .
- (5) Obtain the integer  $x$  from  $C$  such that, for some function  $f(x)$  is maximum.

Notice that the solutions to each of these problems **require you to examine all elements of the container class**.

**The pseudo-code for these problems typically takes the following form**

initialization step;

**for** each item in C

```
{  
    currentItem = current item of C;  
    do something with currentItem;  
}
```

postprocessing step;

**For example, the pseudo-code for **computing maximum** of all element in the container class is:**

1 **int** x=std::numeric\_limits<**int**>::min(); //initialization step

/\*std::numeric\_limits<int>::min() is a method in the C++ Standard Library used to obtain the minimum value representable by the int type, and it is located in the <limits> header file.\*/

2 **for** each item in C

3 {

4     currentItem = current item of C;

5     x=max(currentItem,x); //do something

6 }

7 **return** x; //postprocessing step

■ The C++ implementation of lines 2 and 4 depends on the container class being used.

- For an array **a** of size **n**, the implementation of line 2 and line 4 is:

line 2    `for (int i=0; i<n; i++)`

line 4    `currentItem=a[i];`

- For a **nonempty chain of integers**, the implementation of line 2 and line 4 is:

line 2    `for(ChainNode<int>*ptr=first; ptr!=0;ptr=ptr->link)`

line 4    `currentItem=ptr->data;`

■ If a **container class** is equipped with **iterators**, it can provide **systematic access to the elements of the object**.

## ■ There are other reasons:

- Intuitively, operations such as finding the max element have to be implemented as **member functions of the particular container class** as these operations access private data members of the container class.
- **But there are some drawbacks to this approach for a template class:** For example, we consider the container class `Chain<T>`.
  - Since `Chain<T>` is a template class, even if `T` is initialized to **different type of object**, its operations should be **independent of T**. But this is really hard to do. Operations that are meaningful for one instantiation of `T` may not for another instantiation.

- If a user requires an operation that is not supported by the class `Chain<T>`, he or she would be forced to add that operation to the container class definition. But **it is not a good practice** since the resulting class definition may not consistent with the objectives of the original class. Meanwhile, the number of member functions of `Chain<T>` will become quite large, making the class definition rather unwieldy.
- Even if it is acceptable to add member functions to a class, the user would have to **learn how to sequence through** the elements of container class, which would need learning how the container class is implemented.
- The user can employ **iterators to implement their own functions** depending upon the particular application.

- How to implement an iterator for a container class.
  - Typically, an iterator is implemented as a **public nested class** of the container class.



## C++ Iterators

- An iterator is object that is similar to a pointer to an element of an object (for example, a pointer to an element of an array or chain).
- An iterator permits you to go (or iterate) through the elements of the object one by one.
- The C++ STL defines five categories of iterators: input, output, forward, bidirectional and random access.
  - Except the output iterators, other iterators support the equality operators == and != as well as the dereference operator \* and reference operator ->, but the output iterators does not support reference operator ->.

# A forward Iterator for Chain

- A forward Iterator class for Chain, **ChainIterator**, is a public nested member class of Chain.

```
class ChainIterator { //a public nested member class of Chain.  
private:  
    ChainNode<T>* current;  
public:  
    // typedefs required by C++ for a forward iterator omitted  
  
    // constructor  
    ChainIterator(ChainNode<T>* startNode = 0)  
        { current = startNode; }    //To be continued
```

// dereferencing operators

```
T& operator *() const { return current→data;}
```

/\*When we have an iterator class object obj, the compiler interprets \*obj as obj.operator\*(), i.e., it calls this overloaded function and obtains the T& it returns (which is a reference to the internally managed T object).\*/

```
T* operator →() const { return &current→data;}
```

/\*When we have an iterator class object obj, the compiler interprets obj->member as (obj.operator->())->member. That is, it first calls operator->() to obtain a pointer, and then uses that pointer to access member.

```
// increment
```

```
ChainIterator& operator ++() // preincrement
```

```
{  
    current = current→link;  
    return *this;  
}
```

```
ChainIterator& operator ++(int) // postincrement
```

```
{  
    ChainIterator old = *this;  
    current = current→link;  
    return old;  
} //To be continued
```

// equality testing

```
bool operator !=(const ChainIterator right) const  
    { return current != right.current; }
```

```
bool operator == (const ChainIterator right) const  
    { return current == right.current; }
```

```
}; //end of class definition of ChainIterator
```

- **Additionally, we add the following **public** member functions to **Chain**:**

```
ChainIterator begin() {return ChainIterator(first);}
```

```
ChainIterator end() {return ChainIterator(0);}
```

**which respectively return iterators initialized to the first node of a list and one past the last node.**

# Example

- We may initialize an iterator object **yi** to the start of **a chain of integers y** using the statement:

```
Chain<int>::ChainIterator yi = y.begin();
```

- We can use the following statements to visit every element in a chain of integers **y**:

```
for(Chain<int>::ChainIterator yi = y.begin(); yi != y.end(); y++)  
    visit(*yi);
```

## 4.3.3 Template Class Chain Operations

### ■ InsertBack and Concatenate

To insert and concatenate efficiently **at the end of a chain**, we add a private member **last** to Chain<T>, which points to the last node in the chain.

```
template <class T>
void Chain<T>::InsertBack(const T& e)
{
    if (first) { // nonempty chain
        last→link = new ChainNode<T>(e);
        last = last→link;
    }
    else first = last = new ChainNode<T>(e);
}
```

**The time complexity:  $O(1)$ .**

```
template <class T>
void Chain<T>::Concatenate(Chain<T>& b)
{ // b is concatenate to the end of *this
    if (first)
        { last→link = b.first; last = b.last;}
    else
        { first = b.first; last = b.last; }
    b.first = b.last = 0;
}
```

**The time complexity:  $O(1)$ .**



## ■ Reverse

```
template <class T>
```

```
void Chain<T>::Reverse()
```

```
{
```

```
    ChainNode<T> *current = first,  
                *previous = 0;
```

```
    while (current) {
```

```
        ChainNode<T> *r = previous; ① //r follows previous
```

```
        previous = current; ② //previous follows current
```

```
        current = current→link; ③
```

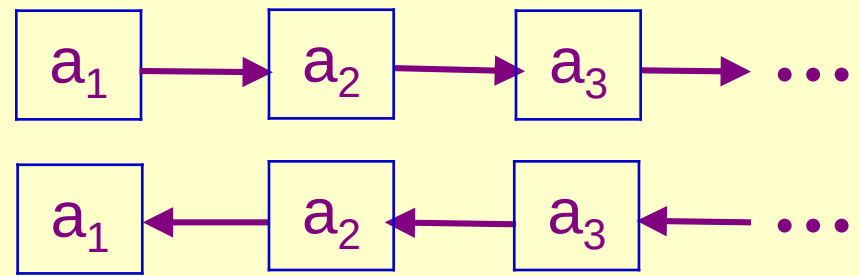
```
        previous→link = r; ④
```

```
    }
```

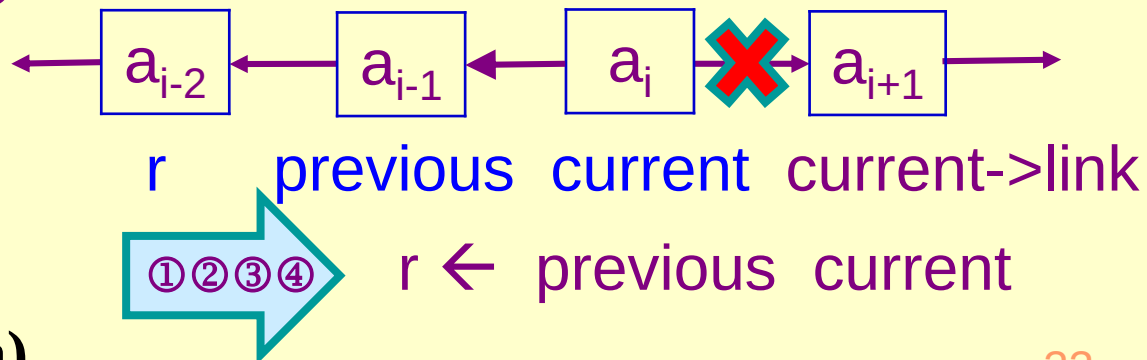
```
    first = previous;
```

```
}
```

The time complexity:  $O(n)$ .



/\*Idea: make the first node of the original remaining chain as the first node (note: not the last node) of the new chain\*/



## ■ Reverse (another style)

```
template <class T>
```

```
void Chain<T>::Reverse()
```

```
{
```

```
    ChainNode<T> *current = first,  
                *previous = 0;
```

```
    while (current) {
```

```
        ChainNode<T> *p = current; ①
```

```
        current = current->link; ②
```

```
        p->link = previous; ③
```

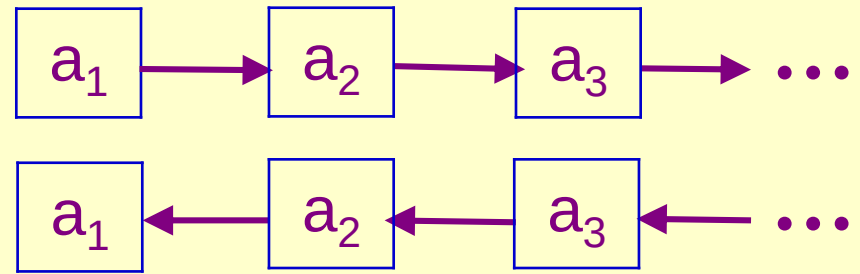
```
        previous = p; ④
```

```
    }
```

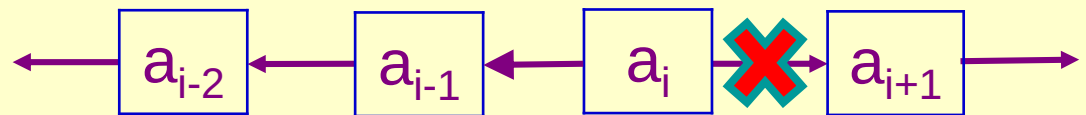
```
    first = previous;
```

```
}
```

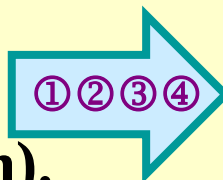
The time complexity:  $O(n)$ .



/\*Idea: make the first node of the original remaining chain as the first node (note: not the last node) of the new chain\*/

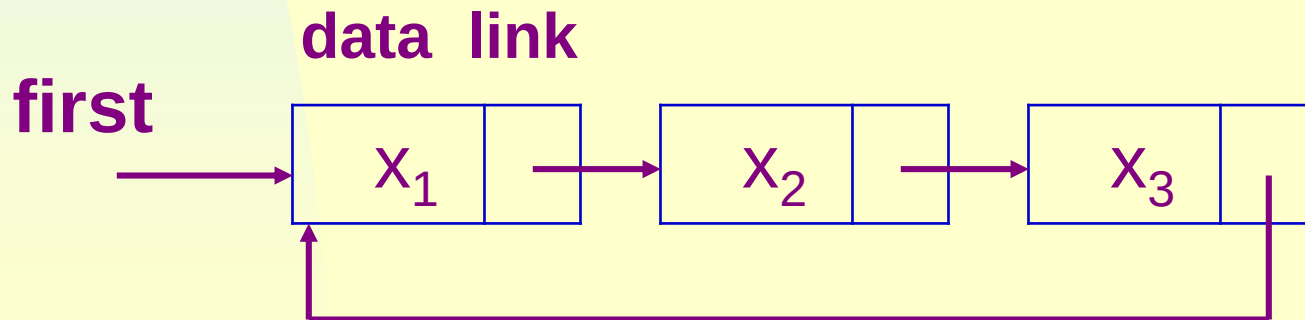


previous    current    current->link  
previous    ← p    current  
previous



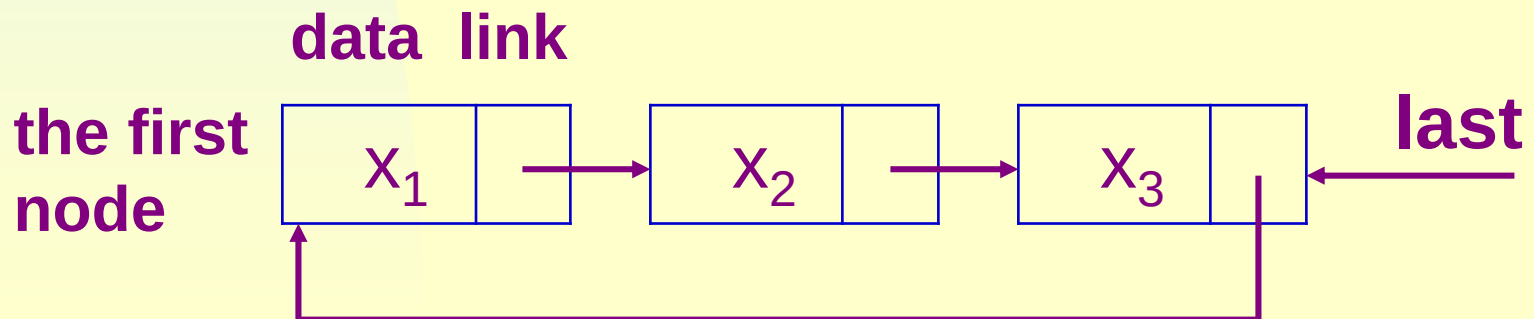
## 4.4 Circular Lists

- Why we need circular list? It will be shown that that there are some advantages. For example, we can visit every node from an arbitrary node.
- A circular list can be obtained by making the **link field of the last node points to the first node** of a chain.



## How to maintain the data member of the circular chain?

- If we only maintain the data member **first** that points the first node  
---Consider **inserting** a new node **at the front** of the circular list:  
We need to **change the link field of the last node**, which requires that we **traverse the entire list** until we find the last node.
- It is more convenient if we maintain the data member **last** that points to the last node.



## Insert at the front:

```
template <class T>
void CircularList<T>::InsertFront(const T& e)
{ // insert the element e at the “front” of the circular list *this,
  // where last points to the last node in the list.
  ChainNode<T>* newNode = new ChainNode<T>(e);
  if (last) { // nonempty list
    newNode→link = last→link;
    last→link = newNode;
  }
  else {
    last = newNode; newNode→link = newNode;
  }
}
```

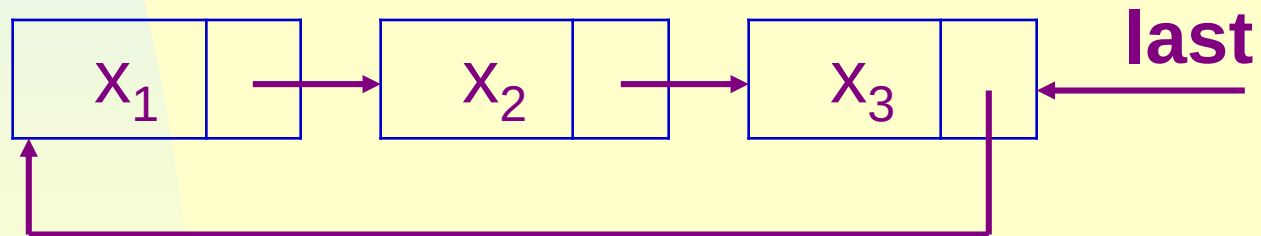
The computing time is  $O(1)$ :

## Insert at the back:

we only need to add the following statement to the if clause of InsertFront.

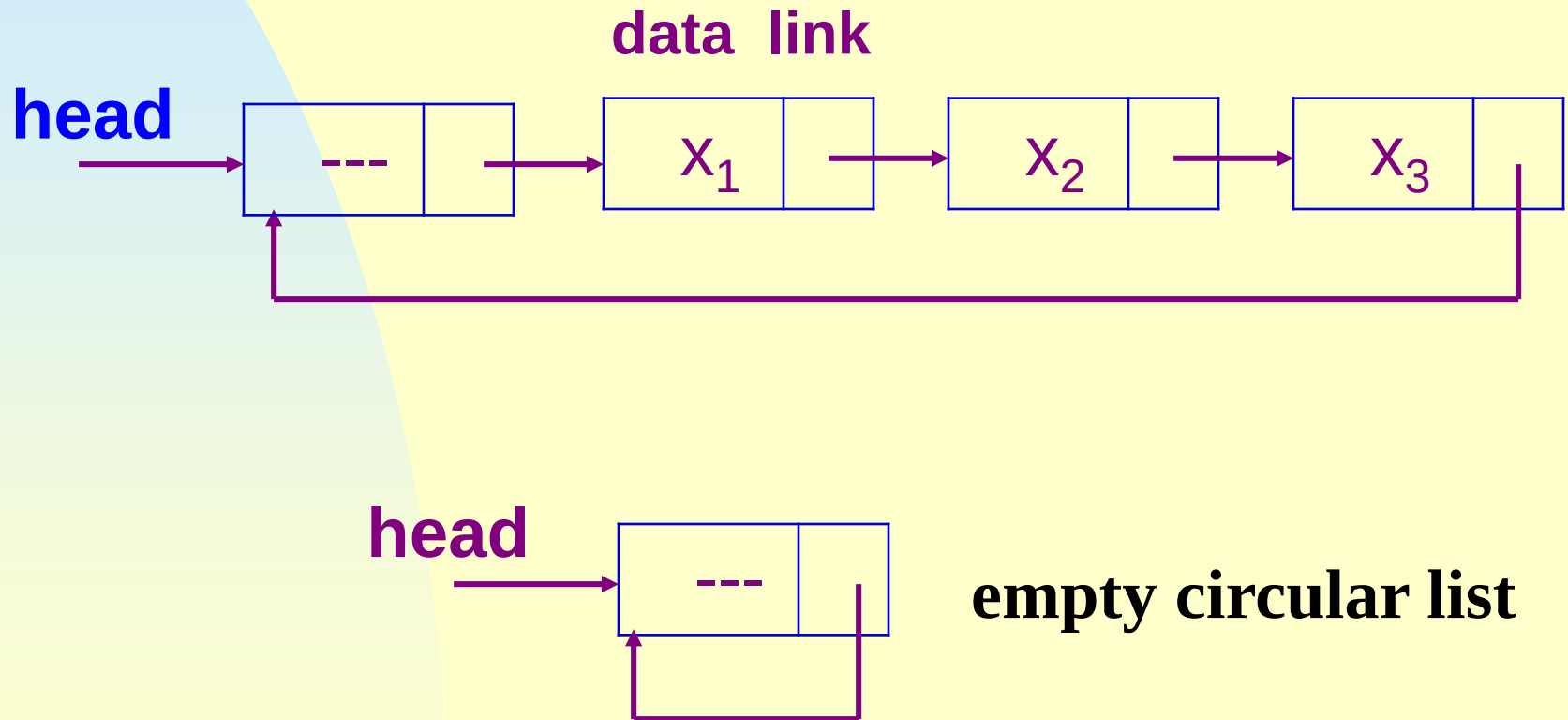
**last = newNode;**

The time complexity is still  $O(1)$ .



## Head node

- To avoid **handling empty list** as a special case, we introduce a dummy **head node**.



## 4.5 Available Space lists

- The destructors for chains and circular lists **take time linear in the length of the chain or list.**
- The run time of these destructors may be reduced to  **$O(1)$**  if we maintain our own chain of **free (or deleted) nodes.**
- When a new node is needed, we may examine our **chain of free nodes:**
  - If this chain is **not empty**, then one of the nodes on it may be made **available** for use.
  - Only when this chain is empty do we need to invoke **new** to create a new node.



- To represent an **available space list (or av list)**, we let **av** be a **static class member** of **CircularList<T>** (or **Chain<T>**) with type **ChainNode<T>\*** that **points to the first node** in our own **chain of free (or deleted) nodes**.

---Initially, **av = 0;**

We shall now use **CircularList<T>::GetNode** or **Chain<T>::GetNode** instead of using **new** to provide a new node:

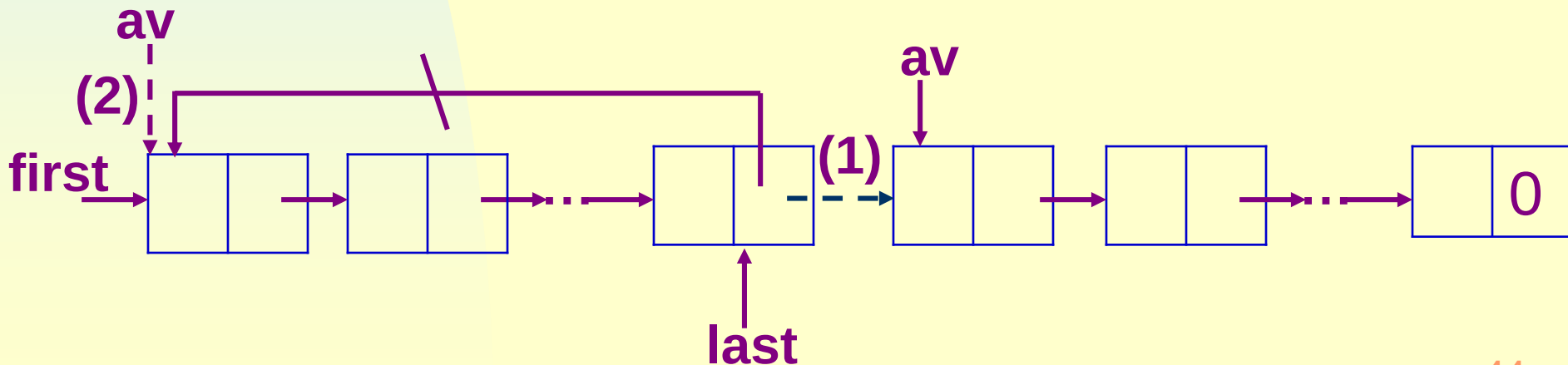
```
template <class T>
ChainNode<T>* CircularList<T>::GetNode( )
{ //provide a node for use
    ChainNode<T> * x;
    if (av) { x = av; av = av→link; x→link=0;}
    else x = new ChainNode<T>;
    return x;
}
```

And we use **CircularList<T>::RetNode** or **CircularList<T>::RetNode** instead of using **delete** to free a node:

```
template <class T>
void CircularList<T>::RetNode(ChainNode<T>*& x)
{ // free the node pointed to by x
    x->link = av;
    av = x;
    x = 0;
}
```

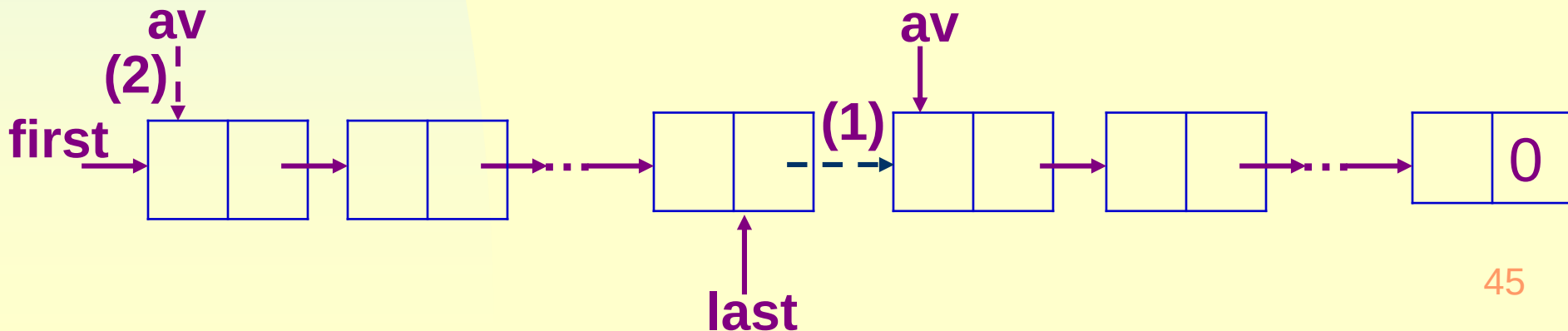
Thus, a **circular list** may be deleted in **O(1)**:

```
template <class T>
void CircularList<T>::~~CircularList()
{ // delete the circular list.
    if (last) {
        ChainNode <T> * first = last→link;
        last→link = av; // (1)
        av = first; // (2)
        last = 0;
    }
}
```

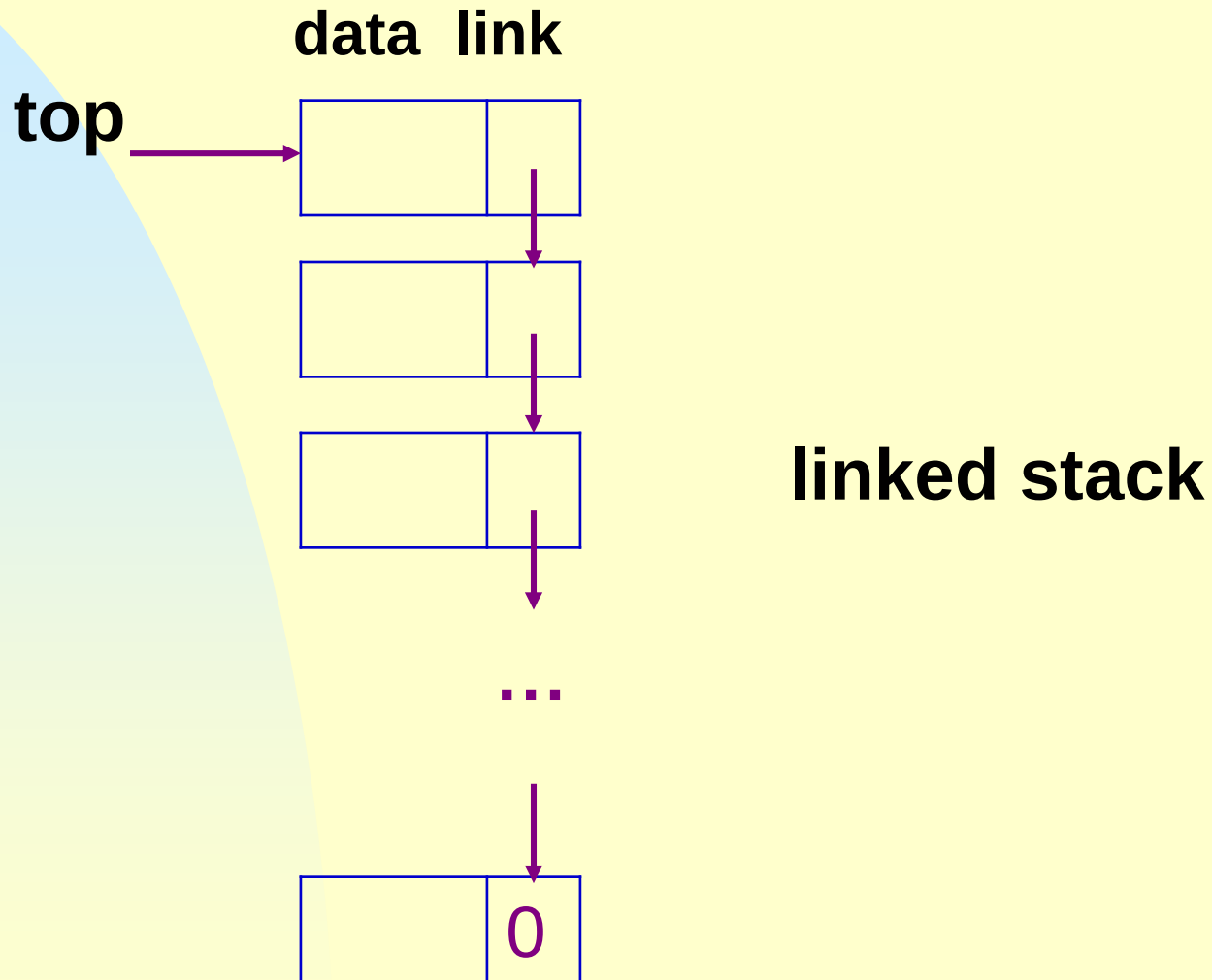


Similarly, a **chain** may be deleted in  $O(1)$  if we know its **first** and **last** nodes:

```
template <class T>
Chain<T>::~~Chain()
{ // delete the chain
    if (first) {
        last→link = av; //(1)
        av = first; //(2)
        last=first = 0;
    }
}
```



## 4.6 Linked Stacks and Queues



# Class definition of linked stack and its operations

Assume the **LinkedStack class** has been declared as **friend of ChainNode<T>**.

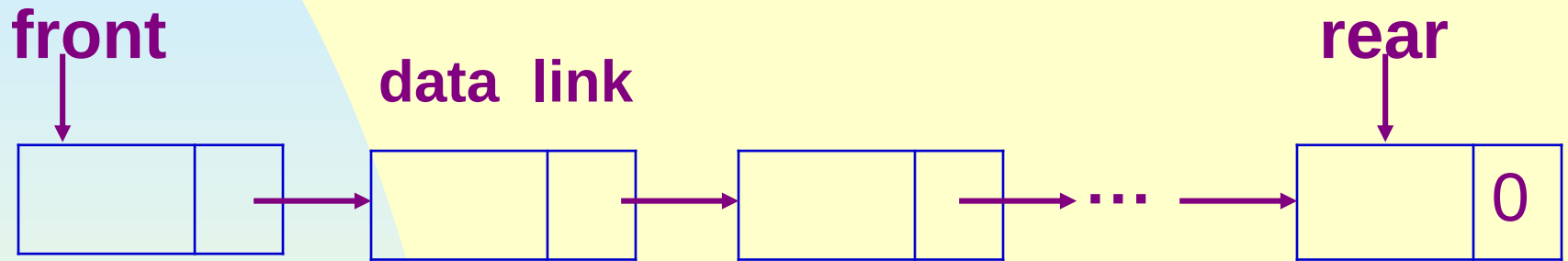
```
template <class T>
class LinkedStack {
public:
    LinkedStack() { top=0;} // constructor initializing top to 0
    // LinkedStack manipulation operations
    ...
private:
    ChainNode<T> *top;
};
```

The functions **IsEmpty** and **Top** are easy to implement, and are omitted.

```
template <class T>
void LinkedStack<T>::Push(const T& e) {
    top = new ChainNode(e, top);
}

template <class T>
void LinkedStack<T>::Pop()
{ // delete top node from the stack.
    if (IsEmpty()) throw "Stack is empty. Cannot delete.";
    ChainNode<T> * delNode = top;
    top = top→link;
    delete delNode;
}
```





**linked queue**

# Class definition of linked queue and its operations

Assume the **LinkedList** class has been declared as **friend** of **ChainNode<T>**.

```
template <class T>
class LinkedList {
public:
    LinkedList() { front=rear=0;} // constructor initializing front
    and rear to 0
    // LinkedList manipulation operations
    ...
private:
    ChainNode<T> *front;
    ChainNode<T> *rear;
};
```

The functions **IsEmpty**, **Front** and **Rear** are easy to implement, and are omitted.

```
template <class T>
```

```
void LinkedQueue<T>::Push(const T& e) {  
    if(IsEmpty()) front=rear= new ChainNode(e, 0);  
    else rear=rear →link=new ChainNode(e, 0);  
}
```

```
template <class T>
```

```
void LinkedQueue<T>::Pop()  
{ // delete first element in queue.  
    if (IsEmpty()) throw “Queue is empty. Cannot delete.”;  
    ChainNode<T> * delNode = front;  
    front = front→link;  
    delete delNode;  
}
```

# 4.7 Polynomials

## 4.7.1 Polynomial Representation

$$A(x) = a_m x^{e_m} + a_{m-1} x^{e_{m-1}} + \dots + a_1 x^{e_1}$$

where  $a_i \neq 0$ ,  $e_m > e_{m-1} > \dots, e_1 \geq 0$

- We shall define a **Polynomial class** to implement polynomials.
- **Definition:** We say that a data object of Type **A** IS-IMPLEMENTED-IN-TERMS-OF a data object of Type **B** if the Type **B** object is **central to the implementation** of Type **A** object.
- This relationship is usually expressed by declaring the **Type B object** as a **data member** of the **Type A object**.
- Here, a polynomial is to be **represented by** a chain, so we say Polynomial object is **IS-IMPLEMENTED-IN-TERMS-OF** Chain object.
- Thus, we make the **chain object poly** be a data member of **Polynomial**.

- For convenience, we shall employ the template class **Chain** which has been defined, and each **ChainNode** will represent a **term**.
- To do this, the list template **T** is instantiated to **struct Term**:

**struct Term**

```
{ // all members of Term are public by default
    int coef;
    int exp;
    Term Set(int c, int e) { coef=c; exp=e; return *this;};
};
```

**class Polynomial {**

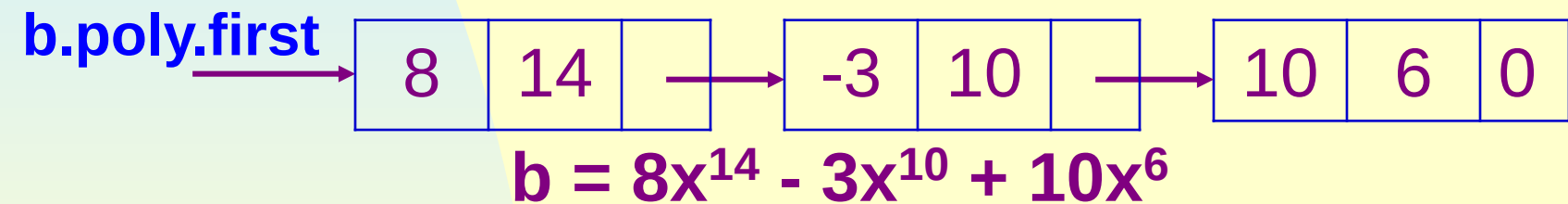
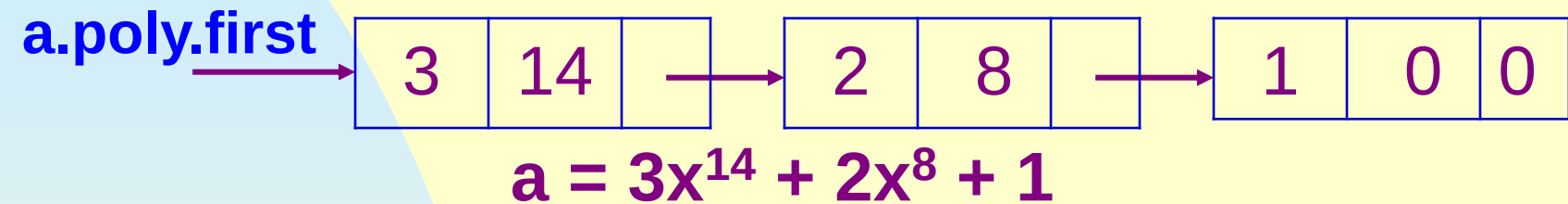
**public:**

// public functions defined here

**private:**

```
    Chain<Term> poly; //Polynomial is IS-IMPLEMENTED-BY Chain ploy
};
```

# Examples of representation of polynomials



## 4.7.2 Adding Polynomials

To add two polynomials **a** and **b**, use the **chain iterators** **ai** and **bi** to move along the terms of a and b.

```
1 Polynomial Polynomial::operator+ (const Polynomial& b) const
2 { // *this (a) and b are added and the sum returned
3   Term temp;
4   Chain<Term>::ChainIterator ai = poly.begin(),
5                               bi = b.poly.begin();
6   Polynomial c;
```

```
7  while (ai != poly.end() && bi != b.poly.end()) { //not null
8      if (ai→exp == bi→exp) {
9          int sum = ai→coef + bi→coef;
10         if (sum) c.poly.InsertBack(temp.Set(sum, bi→exp);
11             ai++; bi++; // to next term
12     }
13     else if (ai→exp < bi→exp) {
14         c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
15         bi++; // next term of b
16     }
17     else {
18         c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
19         ai++; // next term of a
20     }
21 }
```



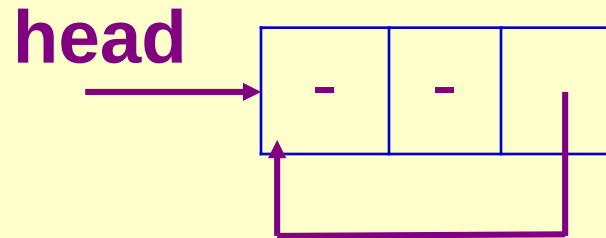
```
22  while (ai != poly.end()) { // copy rest of a
23      c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
24      ai++;
25  }
26  while (bi != b.poly.end()) { // copy rest of b
27      c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
28      bi++;
29  }
30  return c;
31 }
```

## Analysis:

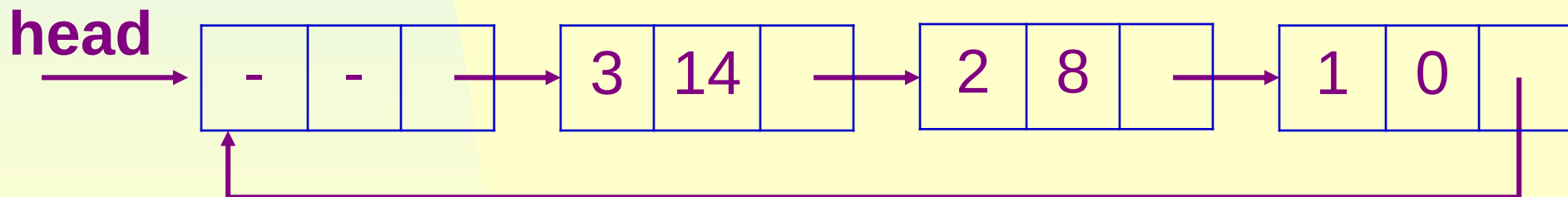
Assume a has **m** terms, b has **n** terms. The computing time is  $O(m+n)$ .

## 4.7.3 Circular List Representation of Polynomials

Polynomials represented by **circular lists with head node** are shown as:



(a) Zero polynomial



(b)  $3x^{14} + 2x^8 + 1$

## Adding operation for polynomials represented circular list with head node

- The **exp of the head node** is set to **-1**.
  - ✓ **Why?** Refer to the following code to answer it.
- Assume the **begin()** function for class **CircularListWithHead** returns an **iterator** that points to the node **head→link**.

```

1 Polynomial Polynomial::operator+(const Polynomial& b) const
2 { // *this (a) and b are added and the sum returned
3   Term temp;
4   CircularListWithHead<Term>::Iterator ai = poly.begin(),
5                                     bi = b.poly.begin();
6   Polynomial c; //assume constructor sets head→exp = -1
7   while (1) {
8     if (ai→exp == bi→exp) {
9       if (ai→exp == -1) return c;
10      int sum = ai→coef + bi→coef;
11      if (sum) c.poly.InsertBack(temp.Set(sum, ai→exp);
12      ai++; bi++; // to next term
13    }

```

```
14  else if (ai→exp < bi→exp) {
15      c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
16      bi++; // next term of b
17  }
18  else {
19      c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
20      ai++; // next term of a
21  }
22 }
23 }
```

- Why is the **exp** of the head node set to **-1**?  
---To push the rest of a or b to the result chain.

## 4.10 Doubly Linked Lists

In singly linked list:

- can move **only in the direction of the links.**
- **deletion an arbitrary node** requires knowing the **preceding node**, it is hard.

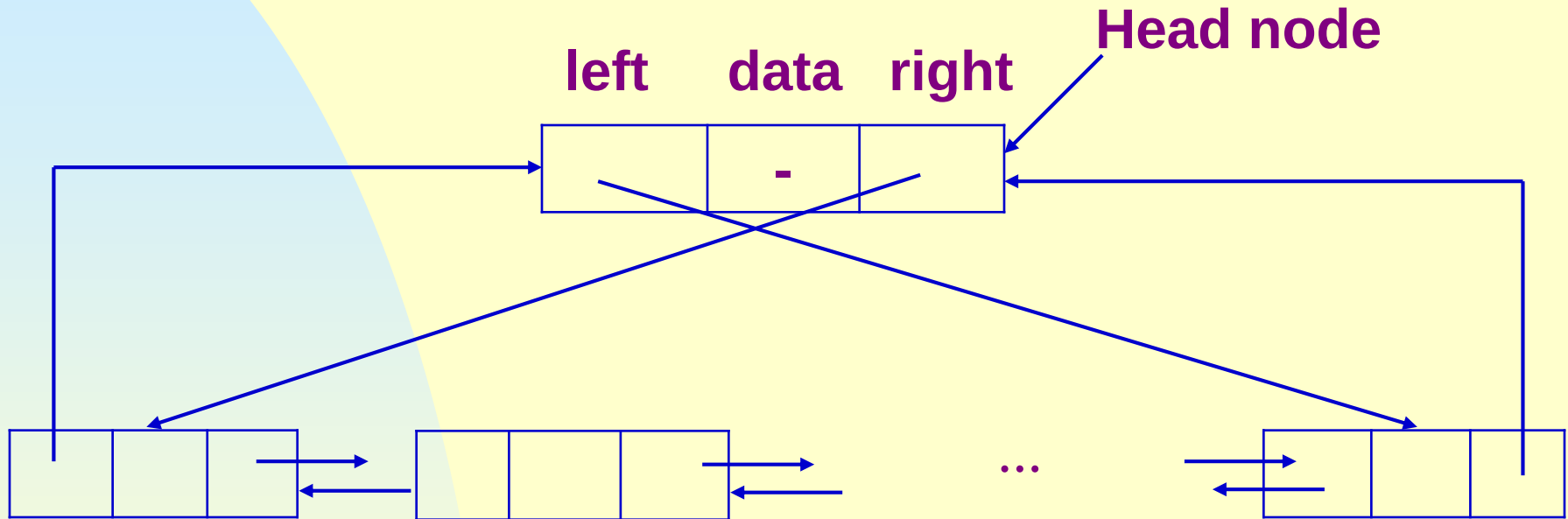
In doubly linked list:

- can move **in both directions easy.**
- **It is easy to know the preceding node for an arbitrary node.**

A node of doubly linked list has 3 field: **data, left and right**



A doubly linked list is usually **circular with head node**.



Suppose  $p$  points to any node, then  
 $p == p \rightarrow \text{left} \rightarrow \text{right} == p \rightarrow \text{right} \rightarrow \text{left}$  (as **right value**)

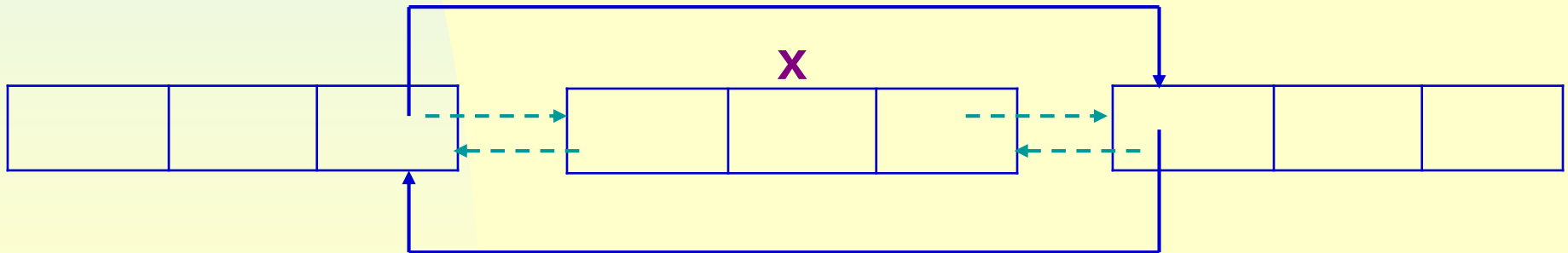
## Program 4.32: Class definition of a doubly linked circular list with head node:

```
class Dbllist;  
  
class DbllistNode{  
friend class Dbllist;  
private:  
    int data;  
    DbllistNode *left, *right;  
};  
  
class Dbllist{  
public:  
    // List manipulation operations  
    ...  
private:  
    DbllistNode *first; // points to head node  
};
```



# Delete

```
void Dbllist::Delete(DblListNode *x )
{
    if(x == first) throw "Deletion of head node not permitted";
    else {
        x→left→right = x→right;
        x→right→left = x→left;
        delete x;
    }
}
```



# Insert

```
void DbList::Insert(DbListNode *p, DbListNode *x )  
{ // insert node p to the right of node x
```

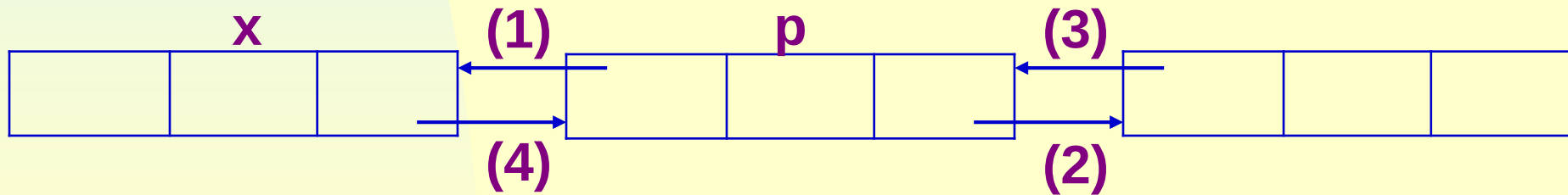
```
    p→left = x;           // (1)
```

```
    p→right = x→right;    // (2)
```

```
    x→right→left = p;     // (3)
```

```
    x→right = p;          // (4)
```

```
}
```



## 4.11 Generalized Lists

**Definition:** A generalized list, is a finite sequence of  $n \geq 0$  elements,  $a_0, a_1, \dots, a_{n-1}$ , where  $a_i$  is either an **atom** or a **list**. The elements  $a_i$ ,  $0 \leq i \leq n-1$ , that are **not atoms** are said to be the **sublists**.

Let  $A = (a_0, \dots, a_{n-1})$  be a list.  $A$  is the name of the list and  $n$  is the length.

By convention, all **list names** are represented by **capital letters**. **Lowercase letters** are used to represent **atoms**. If  $n \geq 1$ , then  $a_0$  is the **head** of  $A$  and  $(a_1, \dots, a_{n-1})$  is the **tail** of  $A$ .

## Some examples of generalized lists:

- (1)  $A=()$ : the null, or empty, list; its length is 0.
- (2)  $B=(a,(b,c))$ : a list of length 2; its first element is the atom  $a$ , and its second element is the linear list  $(b,c)$ .
- (3)  $C=(B,B,())$ : a list of length 3 whose first two elements are the list  $B$ , and the third element is the null list.
- (4)  $D=(a,D)$ : a recursive list of length 2;  $D$  corresponds to the infinite list  $(a,(a,(a,\dots)))$

$\text{head}(B)='a'$ ,  $\text{tail}(B)=((b,c))$ ;  $\text{tail}(B)$  also has a head and tail, which are  $(b,c)$  and  $()$ , respectively.

$\text{head}(C)=B$ , and  $\text{tail}(C)=(B,())$ ;  
 $\text{head}(\text{tail}(C))=B$  and  $\text{tail}(\text{tail}(C))=()$ .

## Notice that:

1. lists may be **shared by other lists** as in example (3), where list B makes up two of the sublists of C;
2. Lists may be **recursive** as in example (4).

# Representation of the Generalized List

Its node can be represented using the **node** structure



**tag=false/true**

This data structure may be defined in C++ as shown in the next slide.

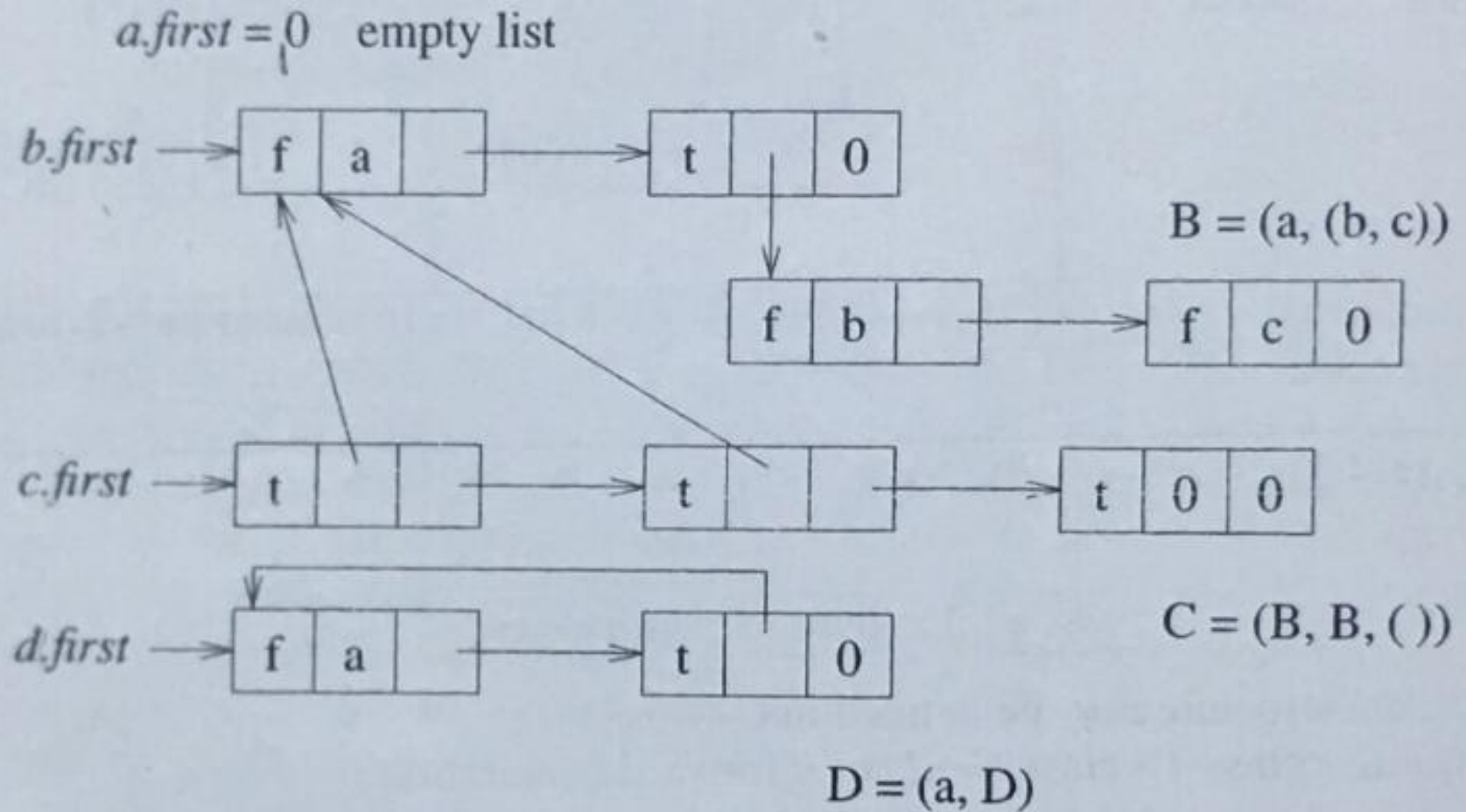
# Class definitions of GenListNode and GenList

```
template<class T>class GenList;  
Template<class T>  
Class GenListNode{  
friend class GenList<T>;  
private:  
    GenListNode<T> *next;  
    bool tag;  
    union{  
        T data;  
        GenListNode<T>*down  
    };  
};
```

```
Template<class T>  
Class GenList{  
public:  
    //List manipulation operations  
private:  
    GenListNode<T>*first;  
};
```



# The representations of the example lists (1) to (4)



**Figure 4.32:** Representation of lists (1) to (4) (page 228); an *f* in the *tag* field represents the value **false**, whereas a *t* represents the value **true**)

## 4.11.2.3 The depth of the nonrecursive lists

The **depth** of the empty generalized list is defined to be zero;

The **depth** of the recursively defined generalize list is infinity;

Otherwise,

$$\text{depth}(s) = \begin{cases} 0 & \text{if } s \text{ is an atom} \\ 1 + \max\{\text{depth}(x_0), \dots, \text{depth}(x_{n-1})\} & \text{if } s \text{ is the list } (x_0, \dots, x_{n-1}), n \geq 1 \end{cases}$$

## Program 4.37: Computing the depth of a list

```
Template<class T>
```

```
int Genlist<T>::Depth() //driver
```

```
{ //Compute the depth of a non-recursive list.
```

```
    return Depth(first);
```

```
}
```

```
Template<class T>
```

```
int Genlist<T>::Depth(GenListNode<T>*s) //workhorse
```

```
{
```

```
    if(!s) return 0; //empty list
```

```
    GenListNode<T>*current=s;
```

```
    int m=0;
```

```
    while(current){
```

```
        if(current->tag) m=max(m, Depth(current->down));
```

```
        current=current->next;
```

```
    }
```

```
    return m+1;
```

```
}
```

# **Exercise(s) and Experiment(s)**

## **Exercise(s):**

- **P183:2; P184:5; P194:4**

## **Experiment(s):**

- **P184:6; P223:6(f,h)**